LAPORAN PRAKTIKUM STRUKTUR DATA

“SINGLE LINKED LIST”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadirat Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-5 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep struktur data Single Linked List dalam bagian pembahasan Linked List.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan penggunaan Single Linked List dalam real Project. Melalui projek projek sederhana yang dilakukan pada praktikum ini membantu mengembangkan pemahaman akan Single Linked List pada program.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhalila

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Program NodeSLL_2511531001 2
2.2.1 Kode Program 2
2.2.2 Penjelasan 2
2.2 Program TambahSLL_2511531001 2
2.3.1 Kode Program 2
2.3.2 Penjelasan 6
2.3.3 Output 8
2.3 Program PencarianSLL_2511531001 8
2.4.1 Kode Program 8
2.4.2 Penjelasan 9
2.4.3 Output 10
2.4 Program HapusSLL_2511531001 10
2.5.1 Kode Program 10
2.5.2 Penjelasan 14
2.5.3 Output 15
BAB III KESIMPULAN 16
DAFTAR PUSTAKA 17

BAB I
PENDAHULUAN

1.1 Latar Belakang

Single Linked List sesuai dengan namanya merupakan salah satu jenis dari sturktur data Linked List. Linked List sendiri sering disebut sebagai senarai berantai. Masing- masing data dalam LinkedList disebut sebagai node yang menempati alokasi memori secara dinamis. Sementara itu, Single Linked List adalah sebuah linked list yang menggunakan sebuah variabel pointer saja untuk menyimpan banyak data. Pada implementasinya single linked list ini dapat diimplementasikan dalam 2 variasi yaitu circular dan nocircular.

Sebagai struktur data yang dinamis dan hanya terhubung dalam satu arah, single linked list biasanya digunakan untuk mengelola data secara efisien tanpa memerlukan memori berurutan seperti array. Selain itu, dengan menggunakan single linked list dapat kita gunakan untuk proses penyisipan dan penghapusan elemen nilai dengan efisien. Single linked list juga memakan memori yang lebih sedikit karena hanya menggunakan satu variabel pointer saja jika dibandingkan dengan double Linked List.

1.2 Tujuan

1. Memahami konsep dasar dari struktur data Single Linked List.
2. Mengidentifikasi penggunaan Single Linked List dalam program.
3. Mengimplementasikan metode bawaan dari Single Linked List.

1.3 Manfaat

1. Memberikan pengalaman langsung dalam mengimplementasikan sturktur data Single Linked List.
2. Melatih ketepatan dalam memilih struktur data untuk menghasilkan program yang lebih efisien dan terstruktur

BAB II
PEMBAHASAN

2.1 Program NodeSLL_2511531001

2.2.1 Kode Program

```
package pekan5_2511531001;
public class NodeSLL_2511531001 {
	//node bagian data
	int data_1001;
	//pointer ke node berikutnya
	NodeSLL_2511531001 next_1001;
	//konstruktor menginisialisasi node dengan data
	public NodeSLL_2511531001 (int data_1001) {
		this.data_1001 = data_1001;
		this.next_1001 = null;

	}

}
```

Kode Program 2.1 : program NodeSLL_2511531001

2.2.2 Penjelasan

Program ini adalah program ADT yang terdiri dari deklarasi variabel data sebagai node dan juga next sebagai pointer untuk node berikutnya dalam program Single Linked List. Program ini juga mencakup konstruktor untuk menginisiasi node node dengan data. Dalam proses inisiasi juga terlihat bahwa kondisi awal dari variabel next sebagai pointer itu adalah kosong atau *null*.

2.2 Program TambahSLL_2511531001

2.3.1 Kode Program

```
package pekan5_2511531001;
```

```
public class TambahSLL_2511531001 {
	public static NodeSLL_2511531001 insertAtFront_1001
(NodeSLL_2511531001 head_1001, int value_1001) {
		NodeSLL_2511531001 new_node_1001 = new
NodeSLL_2511531001 (value_1001);
		new_node_1001.next_1001 = head_1001;
		return new_node_1001;
	}

	// fungsi menambahkan node di akhir SLL
	public static NodeSLL_2511531001 insertAtEnd_1001
(NodeSLL_2511531001 head_1001, int value_1001) {
		// buat sebuah node dengan sebuah nilai
		NodeSLL_2511531001 newNode_1001 = new
NodeSLL_2511531001 (value_1001);

		// jika list kosong maka node jadi head
		if(head_1001 == null) {
			return newNode_1001;
			}
		// simpan head ke variabel semenetara
		NodeSLL_2511531001 last_1001 = head_1001;

		//telusuri ke node akhir
		while (last_1001.next_1001 != null) {
			last_1001 = last_1001.next_1001;
			}

		//ubah pointer
		last_1001.next_1001 = newNode_1001;
		return head_1001;
		}

	static NodeSLL_2511531001 GetNode_1001 (int
data_1001) {
		return new NodeSLL_2511531001 (data_1001);
	}
```

```
	static NodeSLL_2511531001 insertPos_1001
(NodeSLL_2511531001 headNode_1001, int position_1001, int
value_1001) {
		NodeSLL_2511531001 head_1001 = headNode_1001;
		if (position_1001 < 1)
			System.out.println("Invalid Position");
		if (position_1001 == 1) {
			NodeSLL_2511531001 new_node_1001 = new
NodeSLL_2511531001 (value_1001);
			new_node_1001.next_1001 = head_1001;
			return new_node_1001;
			} else {
			while (position_1001-- !=0) {
				if (position_1001 ==1) {
					NodeSLL_2511531001
newNode_1001 = GetNode_1001 (value_1001);
					newNode_1001.next_1001 =
headNode_1001.next_1001;
					headNode_1001.next_1001 =
newNode_1001;
					break;
				}
				headNode_1001 =
headNode_1001.next_1001;
			}
		if (position_1001 != 1)
		System.out.println("Posisi di luar
jangkauan");}
		return head_1001;}

	public static void printList_1001
(NodeSLL_2511531001 head_1001) {
		NodeSLL_2511531001 curr_1001 = head_1001;
			while (curr_1001.next_1001 != null) {

	System.out.print(curr_1001.data_1001 + "-->");
				curr_1001 = curr_1001.next_1001;
```

```
                    }
                    if (curr_1001.next_1001 == null) {

        System.out.print(curr_1001.data_1001);
                    }
                    System.out.println();
        }

        public static void main (String [] args) {
                //buat linked list 2->3->5->6
                NodeSLL_2511531001 head_1001 = new
NodeSLL_2511531001(2);
                head_1001.next_1001 = new
NodeSLL_2511531001(3);
                head_1001.next_1001.next_1001 = new
NodeSLL_2511531001 (5);
                head_1001.next_1001.next_1001.next_1001 = new
NodeSLL_2511531001(6);

                //cetak list asli
                System.out.print("Senarai berantai awal: ");
                printList_1001(head_1001);
                //tambah node baru di depan
                System.out.print("tambah 1 simpul di depan:
");
                int data_1001 = 1;
                head_1001 = insertAtFront_1001(head_1001,
data_1001);
                //cetak update list
                printList_1001(head_1001);

                //tambahkan node abru di belakang
                System.out.print("tambah 1 simpul di
belakang : ");
                int data2_1001 = 7;
                head_1001 = insertAtEnd_1001 (head_1001,
data2_1001);
                //cetak update list
```

```
            printList_1001 (head_1001);

            System.out.print("tambah 1 simpul ke data 4:
");
            int data3_1001 = 4;
            int pos_1001 = 4;
            head_1001 = insertPos_1001
(head_1001,pos_1001,data3_1001);
            // cetak update list
            printList_1001 (head_1001);
        }
}
```

Kode Program 2.2 : program Tambah_SLL

2.3.2 Penjelasan

Program ini dimulai dengan menginisiasi variabel untuk menyimpan node baru yaitu new_node, dan juga menjadikannya sebagai head_1001 jika memenuhi kondisi tertentu. Method pertama dalam program ini adalah insertAtEnd yang mana bertugas untuk memasukkan elemen pada akhir list. Dalam method ini jika memang head dari list masih kosong atau null, maka node yang dimasukkan bukan menjadi new_node lagi melainkan menjadi head dari list yang ada. Setelah itu nilai node yang tersimpan dalam variabel head akan disimpan dalam variabel sementara yaitu last. Dan untuk bisa memasukkan suatu node baru pada akhir list, kita harus menelurusi list sampai ke node terakhir. Penulurusan ini dilakukan dengan pengulangan while pada program Dimana selama node setelah variabel last belum kosong, maka program akan melanjutkan penelusuran dengan memindahkan pointer kepada node setelahnya. Lalu, setelah perulangan while bernilai false atau kondisi tidak terpenuhi lagi, maka barulah pointer yang mengacu dari variabel last akan menjadi cikal bakal new_node yang bisa dimasukkan ke dalam list.

Adapun method yang kedua adalah method insertPos yang berfungsi untuk memasukkan node ke dalam list pada urutan atau posisi tertentu. Pada awalnya head akan disimpan dalam varaibel sementara yaitu headNode dan diuji melalui perulangan if. Di dalam perulangan if, dibutuhkannya variabel position, dan jika position kecil dari 1 maka program akan menampilkan pesan "Invalid Posistion" mengingat posisi dari list array tidak memiliki indeks negative. Dan jika position bernilai 1 maka penambahan new_node pada posisi tersebut dapat dilakukan.

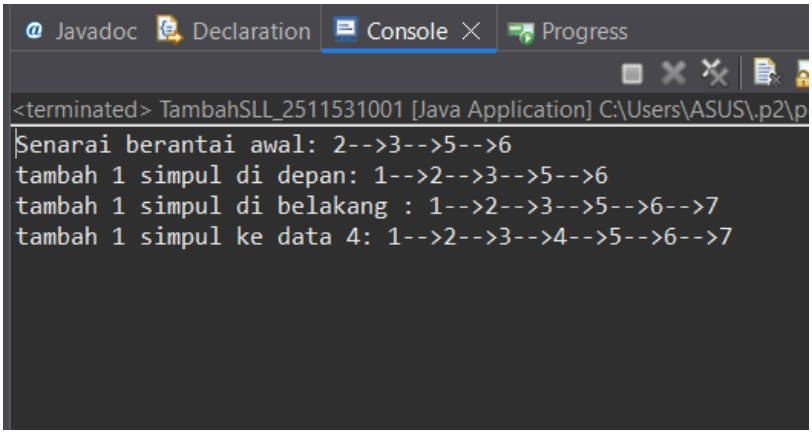
Method ketiga adalah printList dimana nilai head list akan disimpan dalam variabel sementara yaitu curr dan method ini akan menggunakan perulangan while dimana nilai yang disimpan dalam variabel curr masih ada atau bukan kosong, maka program akan menampilkan isi dari varaibel curr dengan tanda panah. Tetapi pada saat perulangan while sudah bernilai false, maka masuk ke dalam percabangan if dimana nilai curr itu adalah kosong maka program akan langsung menampilkan node yang tersimpan sebagai data.

Setelah itu adalah kelas main untuk program ini dimulai dengan inisiasi head sebagai varaibel yang menyimpan ujung dari list. Setelah itu memasukkan nilai node ke dalam list. Untuk menampilkan list asli, kita akan memanggil method printList dengan value yang tersimpan pada varaibel head. Setelahnya untuk menambahkan node abru di depan, kita memanggil method insertAtfront dan juga sebelumnya menginisiasi variabel data dengan nilai 1. Lalu menampilkan list setelah dilakukannya penambahan di depan list.

Menambahkan node baru pada belakang list membutuhkan pemanggilan method insertAtEnd dimana data yang ingin dimasukkan adalah angka 7. Lalu memanggil method printList untuk menampilkan list setelah penambahan node baru di belakang list. Selain itu, program ini juga akan memanggil method insertPos

untuk menambahkan nilai node 4 ke dalam posisi ke 4 dari list, lalu menampilkan list akhir dnegan memanggil method printList.

2.3.3 Output



Gambar 2.1 : Ouput dari program TambahSLL

2.3 Program PencarianSLL_2511531001

2.4.1 Kode Program

```
package pekan5_2511531001;
public class PencarianSLL_2511531001 {
	static boolean searchKey_1001 (NodeSLL_2511531001
head_1001, int key_1001) {
		NodeSLL_2511531001 curr_1001 = head_1001;
		while (curr_1001 !=null) {
			if (curr_1001.data_1001 != key_1001)
				return true;
			curr_1001 = curr_1001.next_1001;
		}
		return false;
		}

	public static void traversal_1001
(NodeSLL_2511531001 head_1001) {
		//mulai dari head
		NodeSLL_2511531001 curr_1001 = head_1001;
		//telusuri sampai pointer null
		while (curr_1001 != null) {
```

```
                    System.out.print(" " +
curr_1001.data_1001);
                    curr_1001 = curr_1001.next_1001;
            }
            System.out.println();
      }

      public static void main (String [] args) {
            NodeSLL_2511531001 head_1001 = new
NodeSLL_2511531001 (14);
            head_1001.next_1001 = new NodeSLL_2511531001
(21);
            head_1001.next_1001.next_1001 = new
NodeSLL_2511531001 (13);
            head_1001.next_1001.next_1001.next_1001 = new
NodeSLL_2511531001 (30);

      head_1001.next_1001.next_1001.next_1001.next_1001 =
new NodeSLL_2511531001 (10);
            System.out.print("Penulusuran SLL :");
            traversal_1001(head_1001);
            //data yang akan dicari
            int key_1001 = 30;
            System.out.print("cari data " + key_1001 + "
= ");
            if (searchKey_1001(head_1001, key_1001))
                  System.out.println("Ketemu");
            else
                  System.out.println("tidak ada");
      }
}
```

Kode Program 2.3 : Pogram PencarianSLL

2.4.2 Penjelasan

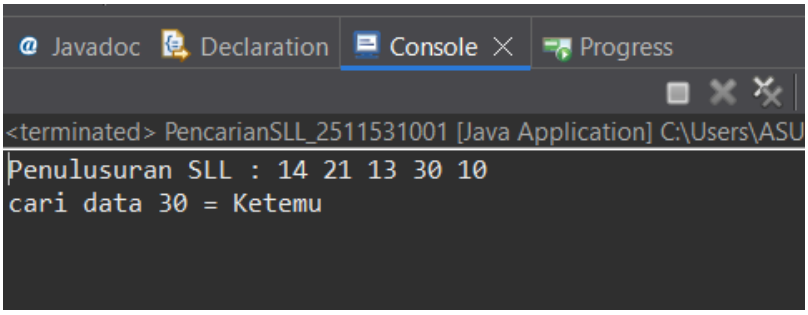
Program ini dimulai dari pendeklarasian method yang bertipe Boolean yaitu searchKey dimana dalam method ini akan terjadi perulangan while selagi variabel curu yang diisi oleh nilai

head itu tidak kosong, maka akan diuji ke percabangan if jika nilai pada curr tidak sama dengan key yang ditentukan maka percabangan akan mengembalikan nilai true, dan akan menyimpan nilai curr sebagai nilai yang curr dengan pointer next.

Setelah itu program ini memiliki method traversal dimana method ini akan melakukan perulangan while dimana jika nilai curr masih belum bernilai kosong maka program akan menampilkan data yang disimpan dalam curr dengan spasi dan mengembalikan nilai curr menjadi sama dengan nilai curr dengan pointer next.

Untuk kelas main diawali dengan memasukkan data node ke dalam list. Lalu program akan memanggil method traversal untuk mencari data key 30 dan akan memanggil method searchKey. Dan jika search key menemukan data yang sesuai maka program akan menampilkan ketemu dan jika tidak ada data yang sesuai pada list yang ada makan program akan menampilkan pesan tidak ketemu.

2.4.3 Output



Gambar 2.2 : Ouput program PencarianSLL

2.4 Program HapusSLL_2511531001

2.5.1 Kode Program

```
package pekan5_2511531001;
public class HapusSLL_2511531001 {
        //fungsi untuk menghapus head
        public static NodeSLL_2511531001 deleteHead_1001
(NodeSLL_2511531001 head_1001) {
                //jika SLL Kosong
```

```
        if (head_1001 == null)
                return null;
        //pindahkan ke node berikutnya
        head_1001 = head_1001.next_1001;
        //return head baru
        return head_1001;
        }

        // fungsi menghapus node terakhir SLL
        public static NodeSLL_2511531001 removeLastNode_1001
(NodeSLL_2511531001 head_1001) {
                //jika list kosong, return null
                if (head_1001 == null) {
                        return null;
                }

                //jika list satu node, hapus node akan return
null
                if (head_1001.next_1001 == null) {
                        return null;
                }
                //temukan node terakhir ke dua
                NodeSLL_2511531001 secondLast_1001 =
head_1001;
                while (secondLast_1001.next_1001.next_1001 !=
null) {
                        secondLast_1001 =
secondLast_1001.next_1001;
                }
                //hapus node terakhir
                secondLast_1001.next_1001 = null;
                return head_1001;
        }

        public static NodeSLL_2511531001 deleteNode_1001
(NodeSLL_2511531001 head_1001, int position_1001) {
                NodeSLL_2511531001 temp_1001 = head_1001;
                NodeSLL_2511531001 prev_1001 = null;
```

```
            //jika linked list null
            if (temp_1001 == null)
                  return head_1001;
            //kasus 1 : head dihapus
            if (position_1001 == 1) {
                  head_1001 = temp_1001.next_1001;
                  return head_1001;
            }

            //kasus 2 : menghapus node di tengah
            //telusuri ke node yang dihapus

            for (int i = 1; temp_1001 != null && i <
position_1001; i++) {
                  prev_1001 = temp_1001;
                  temp_1001 = temp_1001.next_1001;
            }
            //jika ditemukan, hapus node
            if (temp_1001 != null) {
                  prev_1001.next_1001 =
temp_1001.next_1001;
            }else {
                  System.out.println("Data tidak ada");
            }
            return head_1001;
      }

      //fungsi mencetak SLL
      public static void printList_1001
(NodeSLL_2511531001 head_1001) {
            NodeSLL_2511531001 curr_1001 = head_1001;
            while (curr_1001.next_1001 != null) {
                  System.out.print(curr_1001.data_1001+"-
->");
                  curr_1001 = curr_1001.next_1001;
            }
            if (curr_1001.next_1001 == null) {
                  System.out.print(curr_1001.data_1001);
```

```
            }
            System.out.println();
        }

        public static void main (String [] args) {
            //buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
            NodeSLL_2511531001 head_1001 = new
NodeSLL_2511531001 (1);
            head_1001.next_1001 = new NodeSLL_2511531001
(2);
            head_1001.next_1001.next_1001 = new
NodeSLL_2511531001 (3);
            head_1001.next_1001.next_1001.next_1001 = new
NodeSLL_2511531001 (4);

        head_1001.next_1001.next_1001.next_1001.next_1001 =
new NodeSLL_2511531001 (5);

        head_1001.next_1001.next_1001.next_1001.next_1001.ne
xt_1001= new NodeSLL_2511531001 (6);

            //cetak list awal
            System.out.println("list awal : ");
            printList_1001(head_1001);

            //hapus head
            head_1001 = deleteHead_1001 (head_1001);
            System.out.println("List setelah head di
hapus : ");
            printList_1001(head_1001);

            //hapus node terakhir
            head_1001 = removeLastNode_1001 (head_1001);
            System.out.println("List setelah simpul
terakhir di hapus : ");
            printList_1001(head_1001);

            //deleting node position 2
```

```
            int position_1001 = 2;
            head_1001 =
deleteNode_1001(head_1001,position_1001);

            //print list after deletion
            System.out.println("List setelah posisi 2
dihapus: ");
            printList_1001(head_1001);
        }
}
```

Kode Program 2.4 : Program HapusSLL

2.5.2 Penjelasan

Program ini dimulai dari menginisiasi program NodeSLL dan melakukan percabangan if. Jika head dari list tidak bernilai atau null maka program akan mengembalikan nilai null. Setelah itu ada method removeLastNode untuk menghapus nilai node terakhir dari list yang ada. Method ini dimulai dengan pengetesan apakah head dari list kosong atau tidak. Lalu program akan menguji apakah di dalam list hanya berisi 1 node atau tidak. Jika iya program akan mengembalikan nilai null. Namun, jika list terdiri lebih dari 1 node, maka method ini akan menemukan node terakhir ke dua dengan cara menyimpan nilai head ke dalam variabel sementara yaitu secondLast dan dengan perulangan while selama secondLast pada 2 pointer berikutnya tidak kosong, maka nilai yang disimpan ke dalam variabel secondLast adalah nilai dari pointer setelah variabel secondLast. Setelah ditemukan node terakhir, maka barulah method ini akan menjadikan variabel secondLast yang menyimpan nilai node terakhit tersebut menjadi kosong atau bernilai null.

Setelah itu program ini juga memiliki method deleteNode yang berfungsi untuk menghapus node dengan nlai tertentu. Dan program ini juga mencakup method PrintList untuk menampilkan nilai node di dalam List yang sudah dibuat.

Lalu untuk kelas main dalam program ini diawali dengan menambahkan beberapa nilai node ke dalam list. Lalu,, program akan menampilkan list dengan memanggil method printList. Setelah itu program ingin menghapus head dari list yang sudah dibuat dengan memanggil method deleteHead. Begitupun dengan menghapus node terakhir, program akan memanggil method removeLastNode. Lalu program ini juga akan memanggil method deleteNode untuk menghapus node yang ada di posisi ke 2 di dalam list. Terakhir, program akan menampilkan list setelah adanya penghapusan head, akhir node dan juga node ke dua dari list.

2.5.3 Output

```
<terminated> HapusSLL_2511531001 [Java Application] C:\Us
list awal :
1-->2-->3-->4-->5-->6
List setelah head di hapus :
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus :
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.3 : output program HapusSLL

BAB III
KESIMPULAN

Single Linked List adalah struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung melalui pointer satu arah. Setiap node menyimpan dua komponen utama, yaitu data yang dibutuhkan dan referensi (`next`) yang mengarah ke node berikutnya. Berbeda dengan array yang memiliki ukuran statis dan akses indeks langsung, linked list mengalokasikan memori secara otomatis sesuai kebutuhan dan hanya dapat ditelusuri secara berurutan dari simpul awal (head) hingga penanda akhir (null). Sifat linear ini menuntut pemahaman yang tepat dalam memanipulasi pointer, karena setiap kesalahan dalam mengubah referensi dapat menyebabkan terputusnya rantai data atau hilangnya informasi.

Keunggulan utama single linked list terletak pada fleksibilitas pengelolaan memori dan efisiensi saat melakukan operasi penyisipan maupun penghapusan di ujung list, tanpa perlu menggeser elemen lain seperti pada array. Meskipun memiliki keterbatasan berupa tidak adanya akses acak (harus ditelusuri satu per satu) dan kebutuhan memori tambahan untuk menyimpan pointer, struktur ini menjadi fondasi krusial dalam memahami konsep alokasi memori dinamis dan manajemen referensi. Penguasaan logika pointer serta penanganan kondisi list kosong tidak hanya melatih ketelitian dalam pemrograman, tetapi juga membuka jalan untuk mempelajari struktur data yang lebih kompleks di tingkat selanjutnya.

DAFTAR PUSTAKA

[1] D. Candra, "Makalah Single Linked List dalam Bahasa Pemrograman C," *SlideShare*, 2023. [Online]. Available: https://www.slideshare.net/slideshow/makalah-single-linked-list-dalam-bahasadocx/254436610. [Accessed: May 5, 2026].

[2] "Struktur Data: Linked List," *Trivusi*, Jul. 2022. [Online]. Available: https://www.trivusi.web.id/2022/07/struktur-data-linked-list.html. [Accessed: May 5, 2026].

[3] "Single Linked List," *School of Computer Science, Binus University*, Mar. 15, 2017. [Online]. Available: https://socs.binus.ac.id/2017/03/15/single-linked-list/. [Accessed: May 5, 2026

LAPORAN TUGAS

A. Program Pasien_2511531001

```
package pekan5_2511531001; // Deklarasi package untuk
mengorganisir kelas

public class Pasien_2511531001 { // Kelas ADT yang
merepresentasikan node dalam Single Linked List
    // Deklarasi atribut data pasien dan pointer next dengan
suffix NIM
    String nama_1001;
    String penyakit_1001;
    int no_1001;
    Pasien_2511531001 next_1001;

    // Constructor untuk menginisialisasi atribut saat objek node
dibuat
    public Pasien_2511531001 (String nama_1001, String
penyakit_1001, int no_1001) {
        this.nama_1001 = nama_1001;
        this.penyakit_1001 = penyakit_1001;
        this.no_1001 = no_1001;
        this.next_1001 = null;
    }

    // Method getter untuk mengakses nilai atribut privat
    public String getNama_1001() {return nama_1001;}
    public String getPenyakit_1001() {return penyakit_1001;}
    public int getNomor_1001() { return no_1001;}
    public Pasien_2511531001 getNext() {return next_1001;}

    // Method setter untuk mengubah nilai atribut dan
menghubungkan pointer next
    public void setNama_1001 (String nama_1001) {this.nama_1001 =
nama_1001;}
    public void setPenyakit_1001 (String penyakit_1001)
{this.penyakit_1001 = penyakit_1001;}
    public void setNomor_1001 (int no_1001) {this.no_1001 =
no_1001;}
    public void setNext_1001 (Pasien_2511531001 next_1001)
{this.next_1001 = next_1001;}
}
```

B. Program RumahSakit_2511531001

```
package pekan5_2511531001;

import java.util.Scanner;

public class RumahSakit_2511531001 {

    // Counter global untuk nomor antrian auto-increment
    static int counterAntrian_1001 = 0;
```

```
    // Insert at Tail: menambahkan node pasien baru di akhir
linked list dengan prinsip FIFO
    public static Pasien_2511531001
insertAtTail_1001(Pasien_2511531001 head_1001, String nama_1001,
String penyakit_1001) {
        counterAntrian_1001++;
        Pasien_2511531001 newNode_1001 = new
Pasien_2511531001(nama_1001, penyakit_1001, counterAntrian_1001);

        if (head_1001 == null) {
            return newNode_1001;
        }

        Pasien_2511531001 last_1001 = head_1001;
        while (last_1001.getNext() != null) {
            last_1001 = last_1001.getNext();
        }

        last_1001.setNext_1001(newNode_1001);
        return head_1001;
    }

    // Delete Head: menghapus dan menampilkan data pasien
terdepan lalu menggeser head ke node berikutnya
    public static Pasien_2511531001
deleteHead_1001(Pasien_2511531001 head_1001) {
        if (head_1001 == null) {
            System.out.println("Antrian kosong! Tidak ada pasien
untuk dipanggil");
            return null;
        }

        Pasien_2511531001 pasienDipanggil_1001 = head_1001;
        System.out.println("\n=== Pasien Dipanggil ===");
        System.out.println("Nama        : " +
pasienDipanggil_1001.getNama_1001());
        System.out.println("Penyakit    : " +
pasienDipanggil_1001.getPenyakit_1001());
        System.out.println("Nomor Antrian: " +
pasienDipanggil_1001.getNomor_1001());

        return head_1001.getNext();
    }

    // Display List: menelusuri linked list dari head hingga null
dan menampilkan semua data pasien
    public static void printList_1001(Pasien_2511531001
head_1001) {
        if (head_1001 == null) {
            System.out.println("Antrian kosong. Belum ada pasien
terdaftar");
            return;
        }

        System.out.println("\n=== Daftar Antrian ===");
        Pasien_2511531001 curr_1001 = head_1001;
```

```
        while (curr_1001.getNext() != null) {
            System.out.println("[" + curr_1001.getNomor_1001() +
"] " +
                                curr_1001.getNama_1001() + " - " +
                                curr_1001.getPenyakit_1001() + " -
->");
            curr_1001 = curr_1001.getNext();
        }
        if (curr_1001 != null) {
            System.out.println("[" + curr_1001.getNomor_1001() +
"] " +
                                curr_1001.getNama_1001() + " - " +
                                curr_1001.getPenyakit_1001());
        }
    }

    // Search Case-Insensitive: mencari pasien berdasarkan nama
tanpa membedakan huruf besar atau kecil
    public static void searchPatient_1001(Pasien_2511531001
head_1001, String namaCari_1001) {
        if (head_1001 == null) {
            System.out.println("Antrian kosong");
            return;
        }

        Pasien_2511531001 curr_1001 = head_1001;
        boolean ditemukan_1001 = false;

        while (curr_1001 != null) {
            if
(curr_1001.getNama_1001().equalsIgnoreCase(namaCari_1001)) {
                System.out.println("\n=== Pasien Ditemukan ===");
                System.out.println("Nama        : " +
curr_1001.getNama_1001());
                System.out.println("Penyakit    : " +
curr_1001.getPenyakit_1001());
                System.out.println("Nomor Antrian: " +
curr_1001.getNomor_1001());
                ditemukan_1001 = true;
                break;
            }
            curr_1001 = curr_1001.getNext();
        }

        if (!ditemukan_1001) {
            System.out.println("Pasien dengan nama '" +
namaCari_1001 + "' tidak ditemukan");
        }
    }

    // Cek Status Antrian: menghitung total pasien dan
menampilkan informasi pasien terdepan
    public static void checkStatus_1001(Pasien_2511531001
head_1001) {
        if (head_1001 == null) {
```

```
            System.out.println("Status Antrian: KOSONG. Tidak ada
pasien");
            return;
        }

        int total_1001 = 0;
        Pasien_2511531001 curr_1001 = head_1001;
        while (curr_1001 != null) {
            total_1001++;
            curr_1001 = curr_1001.getNext();
        }

        System.out.println("\n=== Status Antrian ===");
        System.out.println("Total Pasien    : " + total_1001);
        System.out.println("Pasien Terdepan : " +
head_1001.getNama_1001() +
                           " (No. " + head_1001.getNomor_1001() +
")");
    }

    // Menu utama: menangani input pengguna dan memanggil method
operasi linked list yang sesuai
    public static void main(String[] args) {
        Scanner input_1001 = new Scanner(System.in);
        Pasien_2511531001 head_1001 = null;
        int pilihan_1001 = 0;

        do {
            System.out.println("\n=== Antrian Rumah Sakit NIM:
2511531001 ===");
            System.out.println("1. Daftarkan Pasien (Insert at
Tail)");
            System.out.println("2. Panggil Pasien (Delete
Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");

            if (input_1001.hasNextInt()) {
                pilihan_1001 = input_1001.nextInt();
                input_1001.nextLine();
            } else {
                System.out.println("Input tidak valid. Silakan
masukkan angka 1-6");
                input_1001.nextLine();
                continue;
            }

            switch (pilihan_1001) {
                case 1:
                    System.out.print("Masukkan Nama Pasien: ");
                    String nama_1001 = input_1001.nextLine();
                    System.out.print("Masukkan Penyakit   : ");
                    String penyakit_1001 = input_1001.nextLine();
```

```
                    head_1001 = insertAtTail_1001(head_1001,
nama_1001, penyakit_1001);
                    System.out.println("Pasien berhasil
didaftarkan! Nomor Antrian: " + counterAntrian_1001);
                    break;
                case 2:
                    head_1001 = deleteHead_1001(head_1001);
                    break;
                case 3:
                    printList_1001(head_1001);
                    break;
                case 4:
                    System.out.print("Masukkan Nama Pasien yang
dicari: ");
                    String cari_1001 = input_1001.nextLine();
                    searchPatient_1001(head_1001, cari_1001);
                    break;
                case 5:
                    checkStatus_1001(head_1001);
                    break;
                case 6:
                    System.out.println("Program selesai. Terima
kasih");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.
Silakan pilih 1-6");
            }
        } while (pilihan_1001 != 6);

        input_1001.close();
    }
}
```

*penjelasan program adalah berupa baris komen pada program.

OUTPUT PROGRAM :

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: budi
Masukkan Penyakit   : demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
```

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: sara
Masukkan Penyakit   : batuk
Pasien berhasil didaftarkan! Nomor Antrian: 2
```

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

=== Daftar Antrian ===
[1] budi - demam -->
[2] sara - batuk
```

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

=== Pasien Dipanggil ===
Nama        : budi
Penyakit    : demam
Nomor Antrian: 1
```

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: sara

=== Pasien Ditemukan ===
Nama        : sara
Penyakit    : batuk
Nomor Antrian: 2
```

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5

=== Status Antrian ===
Total Pasien    : 1
Pasien Terdepan : sara (No. 2)
```

```
=== Antrian Rumah Sakit NIM: 2511531001 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai. Terima kasih
```